

Distributed File-Sharing Project: Code Explanation and DC Concepts

1. Overview

This project implements a fault-tolerant file-sharing system over a private network using distributed computing concepts. The architecture consists of a **Main Server**, a **Backup Server**, and a **Client** application. The Main Server is responsible for uploading and handling file requests, while the Backup Server acts as a standby that receives replication data and heartbeats from the Main Server and takes over if the Main Server fails. The Client application requests file lists and downloads file chunks—automatically switching between servers if necessary.

2. Architecture and Application

Application:

The project is meant for environments (like college networks or private networks) where consistent file distribution is essential. It ensures high availability of resources by:

- Replicating files and related metadata between servers.
- Using heartbeats and automatic failover for fault tolerance.
- Allowing clients to download files even if the primary server is not available.

Alternatives:

- **Peer-to-Peer (P2P) Models:** Instead of a client-server model, a full P2P system (e.g., BitTorrent) could be implemented for pure decentralized sharing.
- **Distributed File Systems:** Solutions such as HDFS (Hadoop Distributed File System) or Ceph provide built-in redundancy, scalability, and fault tolerance.
- **Cloud-based Solutions:** Using cloud storage and replication mechanisms (such as Amazon S3 with cross-region replication) is another alternative.

3. Detailed Code Explanation

3.1. Main Server Code (`main_server.py`)

Key Components:

File Replication:

When a file is added, it is split into chunks and stored in memory. Each file and its metadata (size, download counts) are then replicated to the Backup Server.

Snippet:

```
python
Copy
def replicate_file(self, filename, filepath, chunks):
    try:
        filesize = os.path.getsize(filepath)
        chunk_count = len(chunks)
        with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
            s.connect((BACKUP_IP, REPL_PORT))
            header =
f"replicate|{filename}|{filesize}|{chunk_count}"
            s.sendall(header.encode())
            ack = s.recv(1024) # acknowledgment from backup server
            for chunk in chunks:
                s.sendall(chunk)
            self.log(f"Replicated file '{filename}' to backup server.")
    except Exception as e:
        self.log(f"Replication failed for '{filename}': {e}")
```

- **Purpose:**

This function ensures that the file (split into chunks) is sent to the Backup Server so that it can take over if the Main Server fails.

Dynamic Update Replication:

When a file download begins (first chunk request), the download count is increased, and an update is sent. **Snippet:**

```
python
Copy
def replicate_update(self, filename, download_count):
    try:
        with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
            s.connect((BACKUP_IP, REPL_PORT))
            update_msg = f"update|{filename}|{download_count}"
            s.sendall(update_msg.encode())
            self.log(f"Replicated update for '{filename}': download
count = {download_count}")
    except Exception as e:
```

```
self.log(f"Update replication failed for '{filename}': {e}")
```

- **Purpose:**

This method synchronizes dynamic metadata (download count) with the Backup Server, ensuring consistency in state.

Heartbeat Mechanism:

A background thread sends heartbeat messages to the Backup Server to indicate that the Main Server is alive. **Snippet:**

```
python
Copy
def send_heartbeat(self):
    while True:
        try:
            with socket.socket(socket.AF_INET, socket.SOCK_STREAM)
as hb:
                hb.settimeout(1)
                hb.connect((BACKUP_IP, HEARTBEAT_PORT))
                hb.sendall(b"heartbeat")
        except Exception as e:
            self.log(f"Heartbeat error: {e}")
            time.sleep(2)
```

- **Purpose:**

The heartbeat ensures that the Backup Server knows the Main Server is functioning. If heartbeats stop, the Backup Server will promote itself to active.

Rationale for Choices:

- **TCP Sockets and Threads:**

TCP ensures reliable transfer of data. Threads are used to handle simultaneous connections, replicating real-world distributed systems where multiple clients connect concurrently.

- **In-memory Data Structures:**

Dictionaries are used for rapid prototyping, though for production, persistent storage may be considered.

3.2. Backup Server Code (`backup_server.py`)

Key Components:

Replication Listener:

The Backup Server listens for incoming replication and update messages. **Snippet:**

```
python
Copy
def handle_replication(self, conn, addr):
    try:
        header = conn.recv(1024).decode().strip()
        parts = header.split("|")
        if parts[0] == "replicate" and len(parts) == 4:
            _, filename, filesize_str, chunk_count_str = parts
            filesize = int(filesize_str)
            chunk_count = int(chunk_count_str)
            conn.sendall(b"ack")
            chunks = []
            received = 0
            while received < chunk_count:
                chunk = conn.recv(CHUNK_SIZE)
                if not chunk:
                    break
                chunks.append(chunk)
                received += 1
            if received == chunk_count:
                self.files[filename] = chunks
                self.file_sizes[filename] = filesize
                self.download_counts[filename] = 0 # initial count
                self.log(f"Replicated file '{filename}'
({chunk_count} chunks, {filesize} bytes)")
            else:
                self.log(f"Replication incomplete for '{filename}'
from {addr}.")
            elif parts[0] == "update" and len(parts) == 3:
                _, filename, count_str = parts
                count = int(count_str)
                if filename in self.download_counts:
                    self.download_counts[filename] = count
                    self.log(f"Updated download count for '{filename}'
to {count}")
                else:
                    self.log(f"Received update for unknown file
'{filename}'")
            else:
```

```

        self.log(f"Invalid replication message from {addr}:
{header}")
    except Exception as e:
        self.log(f"Replication error: {e}")
    finally:
        conn.close()

```

- **Purpose:**

This function interprets incoming messages, differentiating between initial file replication and dynamic updates.

Heartbeat Listener and Failover:

The Backup Server receives heartbeat messages on a dedicated port. If heartbeats stop, it promotes itself to active, binding to the client communication port. **Snippet:**

```

python
Copy
def monitor_heartbeat(self):
    while True:
        if (time.time() - self.last_heartbeat) > HEARTBEAT_TIMEOUT
and not self.active:
            self.log("No heartbeat detected. Promoting backup server
to ACTIVE.")
            self.active = True
            self.master.title("MiniTorrent - Backup Server
(Active)")
            threading.Thread(target=self.run_server,
daemon=True).start()
            time.sleep(2)

```

- **Purpose:**

This mechanism guarantees that the Backup Server can take over and serve clients if the Main Server fails.

Rationale for Choices:

- **Dual-Purpose Replication Listener:**

By processing both file replication and dynamic updates on one port, the Backup Server simplifies its communication.

- **Automatic Failover:**

The monitoring function ensures high availability by switching roles when necessary.

3.3. Client Code (**client.py**)

Key Components:

Server Selection and Failover:

The client initially tries the Main Server; if it fails, it refreshes the file list from the Backup Server. **Snippet (abbreviated):**

```
python
Copy
def refresh_file_list(self):
    self.full_file_list = fetch_file_list(MAIN_SERVER_IP)
    if not self.full_file_list:
        self.full_file_list = fetch_file_list(BACKUP_SERVER_IP)
        self.server_ip = BACKUP_SERVER_IP
    else:
        self.server_ip = MAIN_SERVER_IP
```

- **Purpose:**
This approach allows the client to seamlessly switch between servers based on availability, ensuring uninterrupted file downloads.
- **Downloading File Chunks:**
The client requests chunk counts and then downloads file chunks using socket requests.
Purpose:
This part maintains the same process whether the server is Main or Backup, ensuring interface consistency.

Rationale for Choices:

- **Simple Fallback Mechanism:**
The client-side code remains unchanged and functions with multiple servers, reducing maintenance overhead.
- **Threaded Seeding Option:**
The client also has the potential to serve as a file seeder via a dedicated port, paving the way for future P2P functionality.

4. Distributed Computing Concepts Applied

1. Fault Tolerance:

- **Primary-Backup Architecture:**
The backup server constantly mirrors file data and metadata and is ready to take over if the Main Server fails.
- **Heartbeat Mechanism:**
Regular heartbeat messages help in detecting failures quickly.

2. Data Replication and Consistency:

- **File Replication:**
Files are split into chunks and replicated from the Main Server to the Backup Server.
- **Dynamic State Updates:**
Download counts are immediately updated on the Backup Server to maintain consistency.

3. Client Redundancy and Failover:

- **Server Selection:**
The client checks for available servers and switches to the Backup Server if the Main Server is down.
- **Potential P2P Implementation:**
Clients can later serve as seeders to distribute load further.

4. Scalability and Modularity:

- **Threading:**
Both servers use threading to support multiple simultaneous connections, an essential feature of distributed systems.
- **Modular Communication Protocols:**
The replication and update protocols simplify the extension of the system.

5. Conclusion

This document outlines the design, code structure, rationale, and potential alternatives for your distributed file-sharing project. By implementing a fault-tolerant system using replication, heartbeats, and dynamic state updates, the project applies core distributed computing concepts that enhance system resilience, data consistency, and availability.

Feel free to modify the code or the document as needed for further clarity or to incorporate additional features such as persistent storage or a full P2P network.